

Computational Statistics - STAT 575 - HW #2

Alex Towell (atowell@siue.edu)

Problem 1

Derive the E-M algorithm for right-censored normal data with known variance, say $\sigma^2 = 1$. Consider Y_i 's that are i.i.d. from a $N(\theta, 1)$, $i = 1, 2, \dots, n$. We observe $(x_1, \dots, x_n)$ and $(\delta_1, \dots, \delta_n)$, where $x_i = \min(y_i, c)$, and $\delta_i = I(y_i < c)$. Let C be the total number of censored (incomplete) observations. We denote the missing data as $\{Z_i : \delta_i = 0\}$.

Part (a)

Derive the complete log-likelihood, $l(\theta|Y)$.

The unobserved random variates $\{Y_i\}$ are i.i.d. normally distributed,

$$Y_i \sim f_{Y_i}(y|\theta)$$

where

$$f_{Y_i}(y|\theta) = (2\pi)^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(y - \theta)^2\right).$$

The likelihood function is therefore

$$L(\theta|\{y_i\}) = \prod_{i=1}^n (2\pi)^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(y_i - \theta)^2\right) \quad (1)$$

$$= (2\pi)^{-\frac{n}{2}} \exp\left(-\sum_{i=1}^n \frac{1}{2}(y_i - \theta)^2\right). \quad (2)$$

Taking the logarithm of L ,

$$\ell(\theta|\{y_i\}) = \log L(\theta|\{y_i\}) \quad (3)$$

$$= -\frac{n}{2} \log(2\pi) - \frac{1}{2} \sum_{i=1}^n (y_i - \theta)^2 \quad (4)$$

$$= -\frac{n}{2} \log(2\pi) - \frac{1}{2} \sum_{i=1}^n y_i^2 + \theta \sum_{i=1}^n y_i - \frac{n}{2} \theta^2. \quad (5)$$

Anticipating that we will be maximizing the complete log-likelihood with respect to θ , we put any terms that are not a function of θ into k , obtaining the result

$$\ell(\theta|\{y_i\}) = k + \theta \sum_{i=1}^n y_i - \frac{n}{2} \theta^2.$$

Part (b)

Show the conditional expectation

$$E(Y|x, \delta = 1, \theta^{(t)}) = x$$

and

$$E(Y|x, \delta = 0, \theta^{(t)}) = E(Y|Y > x) = \theta^{(t)} + \frac{\phi(x - \mu)}{1 - \Phi(x - \mu)}$$

where ϕ and Φ are pdf and cdf of standard normal.

The distribution of Y given $\delta = 1$, is uncensored and therefore it is given that Y realized the value x . Since the expectation of a constant x is x , that means $E(Y|Y = x) = x$.

If $\delta = 0$, Y is censored, i.e., $Y > x$. To take its expectation, we first need to derive the conditional distribution of Y given $Y > x$ and $\theta^{(t)}$.

The probability $\Pr(Y \leq y|Y > x)$ is given by

$$\Pr(Y \leq y|Y > x) = \Pr(x < Y \leq y) / \Pr(Y > x)$$

which may be rewritten as

$$\Pr(Y \leq y|Y > x) = \frac{F_Y(y|\theta^{(t)}) - F_Y(x|\theta^{(t)})}{1 - F_Y(x|\theta^{(t)})}.$$

where $F_{Y|\theta^{(t)}}$ is the cdf of the normal distribution with $\sigma = 1$ and $\mu = \theta^{(t)}$.

We may rewrite $F_{Y|\theta^{(t)}}$ in terms of the standard normal,

$$F_Y(y|\theta^{(t)}) = \Phi(y - \theta^{(t)}),$$

and thus we may rewrite the conditional distribution of $Y|Y > x$ as

$$\Pr(Y \leq y|Y > x) = \frac{\Phi(y - \theta^{(t)}) - \Phi(x - \theta^{(t)})}{1 - \Phi(x - \theta^{(t)})}$$

and thus after further simplifying, we obtain the cdf of $Y|x$,

$$F_{Y|x}(y|\theta^{(t)}) = 1 - \frac{1 - \Phi(y - \theta^{(t)})}{1 - \Phi(x - \theta^{(t)})}$$

which has a density given by

$$f_Y(y|x, \theta^{(t)}) = \frac{\phi(y - \theta^{(t)})}{1 - \Phi(x - \theta^{(t)})} I(y > x).$$

The expectation of $Y|(x, \theta^{(t)})$ is given by

$$E(Y|x, \theta^{(t)}) = \int_x^\infty y f_Y(y|x, \theta^{(t)}) dy \tag{6}$$

$$= \int_x^\infty y \left(\frac{\phi(y - \theta^{(t)})}{1 - \Phi(x - \theta^{(t)})} \right) dy \tag{7}$$

$$= \frac{1}{1 - \Phi(x - \theta^{(t)})} \int_x^\infty y \phi(y - \theta^{(t)}) dy. \tag{8}$$

Analytically, this is a tricky integration problem. Certainly, it would be trivial to numerically integrate this to obtain a solution, but we seek a closed-form solution.

I searched online, and discovered an interesting way to tackle this integration problem.

Let f and F respectively denote the pdf and cdf of the normally distributed Y . Then,

$$\frac{df}{dy} = -(y - \theta)f(y)$$

and

$$\int_a^b \frac{df}{dy} dy = f(b) - f(a).$$

Then,

$$E(Y|x, \theta^{(t)}) = \frac{1}{1 - F(x)} \int_x^\infty yf(y)dy \quad (9)$$

$$= -\frac{1}{1 - F(x)} \int_x^\infty -(y - \theta^{(t)})f(y)dy + \frac{\theta^{(t)}}{1 - F(x)} \int_x^\infty f(y)dy \quad (10)$$

$$= -\frac{1}{1 - F(x)} \int_x^\infty \frac{df}{dy} dy + \frac{\theta^{(t)}}{1 - F(x)} (1 - F(x)) \quad (11)$$

$$= -\frac{1}{1 - F(x)} (f(\infty) - f(x)) + \theta^{(t)} \quad (12)$$

$$= \frac{f(x)}{1 - F(x)} + \theta^{(t)}. \quad (13)$$

We may rewrite the last line as

$$E(Y|x, \theta^{(t)}) = \theta^{(t)} + \frac{\phi(x - \theta^{(t)})}{1 - \Phi(x - \theta^{(t)})}.$$

Part (c)

Derive the E -step and M -step using parts (a) and (b). Give the updating equation.

E -step

The E -step entails taking the conditional expectation of the complete log-likelihood function $\ell(\theta|\{Y_i\})$ given the observed data $\{x_i\}$ and $\{\delta_i\}$.

$$Q(\theta|\theta^{(t)}) = E_{Y_i|x_i, \delta_i}(\ell(\theta|\{Y_i\})) \quad (14)$$

$$= E_{Y_i|x_i, \delta_i} \left(k + \theta \sum_{i=1}^n Y_i - \frac{n}{2} \theta^2 \right) \quad (15)$$

$$= k - \frac{n}{2} \theta^2 + \theta \sum_{i=1}^n E_{Y_i|x_i, \delta_i}(Y_i). \quad (16)$$

We have already solved the expectation of Y_i given x_i and δ_i . We rewrite Q by substituting $E(Y_i|x_i, \delta_i)$ with its previously found solution,

$$Q(\theta|\theta^{(t)}) = k - \frac{n}{2} \theta^2 + \theta \sum_{i=1}^n \delta_i x_i + (1 - \delta_i) \left(\theta^{(t)} + \frac{\phi(x_i - \theta^{(t)})}{1 - \Phi(x_i - \theta^{(t)})} \right).$$

Letting $C = \sum_{i=1}^n (1 - \delta_i)$, $R := \sum_{i=1}^n \delta_i x_i$, and separating out all terms that are independent of $\theta^{(t)}$,

$$Q(\theta|\theta^{(t)}) = k - \frac{n}{2} \theta^2 + C\theta\theta^{(t)} + R\theta + \theta \sum_{i=1}^n \frac{(1 - \delta_i)\phi(x_i - \theta^{(t)})}{1 - \Phi(x_i - \theta^{(t)})}.$$

M-step

We wish to solve

$$\theta^{(t+1)} = \arg \max_{\theta} Q(\theta | \theta^{(t)}).$$

by solving

$$\left. \frac{dQ(\theta | \theta^{(t)})}{d\theta} \right|_{\theta=\theta^{(t+1)}} = 0,$$

which may be written as

$$-n\theta^{(t+1)} + C\theta^{(t)} + R + \sum_{i=1}^n \frac{(1 - \delta_i)\phi(x_i - \theta^{(t)})}{1 - \Phi(x_i - \theta^{(t)})} = 0.$$

Solving for $\theta^{(t+1)}$ obtains the updating equation

$$\theta^{(t+1)} = \frac{R}{n} + \frac{C}{n}\theta^{(t)} + \frac{1}{n} \sum_{i=1}^n \frac{(1 - \delta_i)\phi(x_i - \theta^{(t)})}{1 - \Phi(x_i - \theta^{(t)})}.$$

where

$$R := \sum_{i=1}^n \delta_i x_i$$

and

$$C := \sum_{i=1}^n (1 - \delta_i).$$

Part (d)

Use your algorithm on the V.A. data to find the MLE of μ . Take the log of the event times first and standardize by sample standard deviation. You may simply use the censored data sample mean as your starting value.

In the following R code, we implement the updating equation derived in the previous step. We encapsulate the procedure into a function that takes its arguments in the form of a censored set, uncensored set, starting value ($\theta^{(1)}$), and an ϵ value to control stopping condition.

```
# assuming the uncensored and censored data are distributed normally,
# we use the EM algorithm to derive an estimator given censored and uncensored
# data.
mean_normal_censored_estimator_em <- function(uncensored,censored,theta,eps=1e-6,debug=T)
{
  dev <- sd(log(c(uncensored,censored)))
  censored <- log(censored) / dev
  uncensored <- log(uncensored) / dev
  theta <- log(theta) / dev

  n <- length(censored) + length(uncensored)
  C <- length(censored)
  R <- sum(uncensored)

  s <- function(theta)
  {
    sum <- 0
    for (i in 1:C)
    {
```

```

    num <- dnorm(censored[i],mean=theta,sd=1)
    denom <- 1-pnorm(censored[i],mean=theta,sd=1)
    sum <- sum + (num / denom)
  }
  sum
}

i <- 1
repeat
{
  theta.new <- R/n + C/n * theta + (1/n)*s(theta)
  if (debug==T) { cat("theta[", i, "] =",theta," ", theta["", i+1, "] =",theta.new,"\n") }
  if (abs(theta.new - theta) < eps)
  {
    theta <- theta.new * dev
    theta <- exp(theta)
    return(theta)
  }
  i <- i + 1
  theta <- theta.new
}
}

```

We apply this procedure to the indicated data set.

```

library(MASS) # has VA data
VAs <- subset(VA,prior==0)
censored <- VAs$status == 0
censored_xs <- VAs[censored,c("stime")]
uncensored_xs <- VAs[!censored,c("stime")]

mu <- mean(uncensored_xs)
cat("mean of the uncensored sample is ", mu, ".")

## mean of the uncensored sample is 112.1648 .

sol <- mean_normal_censored_estimator_em(uncensored_xs,censored_xs,mu)

## theta[ 1 ] = 3.857928 , theta[ 2 ] = 3.424258
## theta[ 2 ] = 3.424258 , theta[ 3 ] = 3.415443
## theta[ 3 ] = 3.415443 , theta[ 4 ] = 3.415286
## theta[ 4 ] = 3.415286 , theta[ 5 ] = 3.415283
## theta[ 5 ] = 3.415283 , theta[ 6 ] = 3.415283

sol

## [1] 65.2625

```

We see that our estimate of θ is $\hat{\theta} = 65.2624985$. (The θ before transforming it to the appropriate scale was 3.415283.)

This mean is somewhat lower than anticipated, which makes me suspect something is wrong with my updating equation. If I have the time, I will revisit it.

Problem 2

Part (a)

There are $N = 1500$ gay men in the survey sample where X_i denotes the i -th persons response to the number of risky sexual encounters he had in the previous 30 days. Thus, we observe a sample $\vec{X} = (X_1, X_2, \dots, X_N)$.

We assume there are 3 groups in the population, denoted by $z = 1$, $t = 2$, and $p = 3$. Group 1 members report 0 risky sexual encounters regardless of the truth where the probability of being a member of group 1 is denoted by α ,

Group 2 members accurately report risky sexual encounters and represent typical behavior where the probability of being a member of group 2 is denoted by β . We assume this group's number of sexual encounters follows a poisson with mean μ .

Group 3 members accurately report risky sexual encounters and represent high-risk behavior where the probability of being a member of group 3 is $\gamma = 1 - \alpha - \beta$. We assume this group's number of sexual encounters follows a poisson with mean λ .

This represents a finite mixture model with a pdf

$$X_i \sim f(x|\vec{\theta}) = \alpha I(x=0) + \beta \text{POI}(x|\mu) + (1 - \alpha - \beta) \text{POI}(x|\lambda)$$

with a parameter vector

$$\vec{\theta} = (\alpha, \beta, \mu, \lambda)'$$

Let the uncertain group that the i -th person belongs to be denoted by Z_i . If we observe group membership data, $X_i|Z_i = z_i$, then

$$X_i|Z_i = 1 \sim I(x=0), \tag{17}$$

$$X_i|Z_i = 2 \sim \text{POI}(\mu), \tag{18}$$

$$X_i|Z_i = 3 \sim \text{POI}(\lambda), \tag{19}$$

where

$$Z_i \sim f_{Z_i}(z_i|\vec{\theta}) = \Pr(Z_i = z_i) = \begin{cases} \alpha & z_i = 1, \\ \beta & z_i = 2, \\ \gamma = 1 - \alpha - \beta & z_i = 3, \end{cases}$$

and thus

$$f_{X_i, Z_i}(x_i, z_i|\vec{\theta}) = \alpha I(z_i = 1) + \beta \text{POI}(\mu) I(z_i = 2) + (1 - \alpha - \beta) \text{POI}(\lambda) I(z_i = 3).$$

The *complete* likelihood function is thus given by

$$\mathcal{L}(\vec{\theta}|\vec{X}, \vec{Z}) = \prod_{i=1}^N f_{X_i, Z_i}(x_i, z_i|\vec{\theta}),$$

which may be rewritten as

$$\mathcal{L}(\vec{\theta}|\vec{X}, \vec{Z}) = \left(\prod_{\{i|z_i=1\}} \alpha I(x_i=0) \right) \left(\prod_{\{i|z_i=2\}} \beta \frac{\mu^{x_i} e^{-\mu}}{x_i!} \right) \left(\prod_{\{i|z_i=3\}} \gamma \frac{\lambda^{x_i} e^{-\lambda}}{x_i!} \right).$$

We wish to rewrite this so that the data is explicitly represented. First, we do the transformation

$$\mathcal{L}(\vec{\theta}|\vec{X}, \vec{Z}) = \left(\prod_{\{i|z_i=1, x_i=0\}} \alpha \right) \prod_{k=0}^{16} \left(\prod_{\{i|z_i=2, x_i=k\}} \beta \frac{\mu^k e^{-\mu}}{k!} \right) \prod_{k=0}^{16} \left(\prod_{\{i|z_i=3, x_i=k\}} \gamma \frac{\lambda^k e^{-\lambda}}{k!} \right).$$

We let $n_{a,b}$ denote the (unobserved) cardinality of $\{i|z_i = a, x_i = b\}$, thus

$$\mathcal{L}(\vec{\theta}|\{n_{j,k}\}) = \alpha^{n_{1,0}} \prod_{k=0}^{16} \beta^{n_{2,k}} \frac{\mu^{kn_{2,k}} e^{-\mu n_{2,k}}}{(k!)^{n_{2,k}}} \prod_{k=0}^{16} \gamma^{n_{3,k}} \frac{\lambda^{kn_{3,k}} e^{-\lambda n_{3,k}}}{(k!)^{n_{3,k}}}$$

is the complete likelihood. The complete log-likelihood is thus

$$\ell(\vec{\theta}|\{n_{j,k}\}) = n_{1,0} \log \alpha + \sum_{k=0}^{16} \log \left(\beta^{n_{2,k}} \frac{\mu^{kn_{2,k}} e^{-\mu n_{2,k}}}{(k!)^{n_{2,k}}} \right) + \sum_{k=0}^{16} \log \left(\gamma^{n_{3,k}} \frac{\lambda^{kn_{3,k}} e^{-\lambda n_{3,k}}}{(k!)^{n_{3,k}}} \right)$$

which simplifies to

$$\begin{aligned} \ell(\vec{\theta}|\{n_{j,k}\}) = n_{1,0} \log \alpha + \sum_{k=0}^{16} n_{2,k} (\log \beta + k \log \mu - \mu - \log k!) + \\ n_{3,k} (\log \gamma + k \log \lambda - \lambda - \log k!). \end{aligned} \quad (20)$$

Anticipating taking $\frac{d\ell}{d\vec{\theta}}$ to solve for the maximum of the log-likelihood, we remove any terms that are not a function of $\vec{\theta}$, resulting in the kernel

$$\ell(\vec{\theta}|\{n_{j,k}\}) = n_{1,0} \log \alpha + \sum_{k=0}^{16} \{n_{2,k} (\log \beta + k \log \mu - \mu) + n_{3,k} (\log \gamma + k \log \lambda - \lambda)\}.$$

E-step

The conditional expectation to solve in the EM algorithm is given by

$$Q(\vec{\theta}|\vec{\theta}^{(t)}) = E(\ell(\vec{\theta}))$$

where $\{n_{k,j}\}$ are random and $\{n_j\}$ and $\vec{\theta}^{(t)}$ are given. We rewrite this as

$$Q(\vec{\theta}|\vec{\theta}^{(t)}) = E \left(n_{1,0} \log \alpha + \sum_{k=0}^{16} \{n_{2,k} (\log \beta + k \log \mu - \mu) + n_{3,k} (\log \gamma + k \log \lambda - \lambda)\} \right).$$

Using the linearity of expectations, we rewrite the above to

$$Q(\vec{\theta}|\vec{\theta}^{(t)}) = E(n_{1,0}) \log \alpha + \sum_{k=0}^{16} \{E(n_{2,k}) (\log \beta + k \log \mu - \mu) + E(n_{3,k}) (\log \gamma + k \log \lambda - \lambda)\}$$

given $\{n_j\}$ and $\theta^{(t)}$.

Consider $E(n_{2,k}|\{n_j\}, \theta^{(t)})$. To solve this expectation, we must first derive the distribution of $n_{2,k}$.

Suppose $x_j = k$, then probability that the j -th person belongs to group 2 is given by

$$\Pr(Z_j = 2|x_j = k) = \Pr(Z_j = 2) \Pr(x_j = k|Z_j = 2) / \Pr(x_j = k).$$

We note that $\Pr(x_j = k)$ is equivalent to $\pi_k(\vec{\theta})$, $\Pr(Z_j = 2)$ is the definition of β , and $\Pr(x_j = k|Z_j = 2)$ is $f_{X_j|Z_j}(k|Z_j = 2) = \text{POI}(k|\mu)$.

Making the substitutions yields the result

$$t_k(\vec{\theta}) = \Pr(Z_j = 2|x_j = k) = \beta \text{POI}(k|\mu) / \pi_k(\vec{\theta}).$$

Assuming $\{X_i\}$ are i.i.d., observe that $k \neq 0$, the distribution of $n_{2,k}$ given $n_k, \theta^{(t)}$ is binomial distributed with a probability of success $t_k(\vec{\theta}^{(t)})$. Thus,

$$\mathbb{E}(n_{2,k}) = n_k t_k(\vec{\theta}^{(t)}).$$

The same logic holds for $n_{3,k}$ and $n_{1,0}$, and thus

$$\mathbb{E}(n_{3,k}) = n_k p_k(\vec{\theta}^{(t)})$$

and

$$\mathbb{E}(n_{1,0}) = n_0 z_0(\vec{\theta}^{(t)}),$$

which means

$$Q(\vec{\theta}|\vec{\theta}^{(t)}) = n_0 z_0(\vec{\theta}^{(t)}) \log \alpha + \sum_{k=0}^{16} \left\{ n_k t_k(\vec{\theta}^{(t)}) (\log \beta + k \log \mu - \mu) + n_k p_k(\vec{\theta}^{(t)}) (\log \gamma + k \log \lambda - \lambda) \right\}$$

M-step

We wish to solve

$$\vec{\theta}^{(t+1)} = \arg \max_{\vec{\theta}} Q(\vec{\theta}|\vec{\theta}^{(t)}).$$

by solving

$$\nabla Q(\vec{\theta}|\vec{\theta}^{(t)}) \Big|_{\vec{\theta}=\vec{\theta}^{(t+1)}} = \vec{0}.$$

We use the Lagrangian to impose the restriction $\alpha + \beta + \gamma = 1$, thus we seek to perform the constrained maximization of

$$Q_l(\vec{\theta}, c|\vec{\theta}^{(t)}) = Q(\vec{\theta}|\vec{\theta}^{(t)}) + c(1 - \alpha - \beta - \gamma).$$

Thus, when we solve for α ,

$$\frac{\partial Q_l}{\partial \alpha} = \frac{n_0 z_0(\theta^{(t)})}{\alpha} - c = 0,$$

we get the result

$$\alpha^{(t+1)} = \frac{1}{c} n_0 z_0(\theta^{(t)}).$$

Similar results hold for β and γ , obtaining

$$\beta^{(t+1)} = \frac{1}{c} \sum_{k=0}^{16} n_k t_k(\theta^{(t)}).$$

and

$$\gamma^{(t+1)} = \frac{1}{c} \sum_{k=0}^{16} n_k p_k(\theta^{(t)}).$$

This does not look too promising until we realize that

$$n_0 z_0(\theta^{(t)}) + \sum_{k=0}^{16} n_k t_k(\theta^{(t)}) + \sum_{k=0}^{16} n_k p_k(\theta^{(t)}) = N.$$

Thus, $c(\alpha^{(t)} + \beta^{(t)} + \gamma^{(t)}) = N$, which means $c = N$ since $\alpha^{(t)} + \beta^{(t)} + \gamma^{(t)} = 1$. Making this substitution obtains the result

$$\alpha^{(t+1)} = \frac{1}{N} n_0 z_0(\theta^{(t)}) \quad (21)$$

$$\beta^{(t+1)} = \frac{1}{N} \sum_{k=0}^{16} n_k t_k(\theta^{(t)}) \quad (22)$$

$$\gamma^{(t+1)} = \frac{1}{N} \sum_{k=0}^{16} n_k p_k(\theta^{(t)}). \quad (23)$$

Solving an estimator for μ at iteration $(t+1)$,

$$\left. \frac{\partial Q_l}{\partial \mu} \right|_{\mu=\mu^{(t+1)}} = 0 \quad (24)$$

$$\sum_{k=0}^{16} n_k t_k(\theta^{(t)}) (k/\mu^{(t+1)} - 1) = 0 \quad (25)$$

$$\frac{1}{\mu^{(t+1)}} \sum_{k=0}^{16} n_k t_k(\theta^{(t)}) k = \sum_{k=0}^{16} n_k t_k(\theta^{(t)}) \quad (26)$$

$$\mu^{(t+1)} = \frac{\sum_{k=0}^{16} k n_k t_k(\theta^{(t)})}{\sum_{k=0}^{16} n_k t_k(\theta^{(t)})}. \quad (27)$$

The same derivation essentially follows for λ , and thus

$$\lambda^{(t+1)} = \frac{\sum_{k=0}^{16} k n_k p_k(\theta^{(t)})}{\sum_{k=0}^{16} n_k p_k(\theta^{(t)})}.$$

Part (b)

Estimate the parameters of the model, using the observed data.

```
# we observe n = (n0,n1,...,n16)
ns <- c(379,299,222,145,109,95,73,59,45,30,24,12,4,2,0,1,1)
N <- sum(ns)

# theta := (alpha, beta, mu, lambda)'
# note that there is an implicit parameter gamma s.t.
# alpha + beta + gamma = 1
# the initial value assumes each category z, t, or p
# is equally probable, and so we let
# (alpha^(0),beta^(0)) = (1/3,1/3)
# and mu^(0) and lambda^(0) are just arbitrarily chosen to be 2 and 3,
# with the insight that group 3 is more risky than group 2.
theta <- c(1/3,1/3,2,3)

# theta := (alpha, beta, mu, lambda)
Pi <- function(i,theta)
{
  res <- 0
  if (i == 0)
    res <- theta[1]
```

```

res <- res + theta[2] * theta[3]^i * exp(-theta[3])
res <- res + (1 - theta[1] - theta[2]) * theta[4]^i * exp(-theta[4])
res
}

z0 <- function(theta)
{
  theta[1] / Pi(0,theta)
}

t <- function(i,theta)
{
  theta[2] * theta[3]^i * exp(-theta[3]) / Pi(i,theta)
}

p <- function(i,theta)
{
  (1-theta[1] - theta[2]) * theta[4]^i * exp(-theta[4]) / Pi(i,theta)
}

# update algorithm, based on EM algorithm
update <- function(theta,ns)
{
  # note: n0 := ns[1] instead of ns[0] since R does not use zero-based indexes
  alpha <- ns[1] * z0(theta) / N
  beta <- 0
  mu_num <- 0
  mu_denom <- 0

  lam_num <- 0
  lam_denom <- 0

  for (i in 0:16)
  {
    ti <- t(i,theta)
    pi <- p(i,theta)

    beta <- beta + ns[i+1] * ti

    mu_num <- mu_num + i * ns[i+1] * ti
    mu_denom <- mu_denom + ns[i+1] * ti

    lam_num <- lam_num + i * ns[i+1] * pi
    lam_denom <- lam_denom + ns[i+1] * pi
  }

  beta <- beta / N
  mu <- mu_num / mu_denom
  lam <- lam_num / lam_denom

  c(alpha,beta,mu,lam)
}

```

```

em <- function(theta,ns,steps=10000,debug=T)
{
  for(i in 1:steps)
  {
    theta = update(theta,ns)
    if (debug==T)
    {
      if (i %% 1000 == 0) { cat("iteration =",i," theta = (",theta,")'\n") }
    }
  }
  theta
}

# solution theta = (alpha, beta, mu, lambda)
sol <- em(theta,ns,10000,T)

## iteration = 1000  theta = ( 0.1221661 0.5625419 1.467475 5.938889 )'
## iteration = 2000  theta = ( 0.1221661 0.5625419 1.467475 5.938889 )'
## iteration = 3000  theta = ( 0.1221661 0.5625419 1.467475 5.938889 )'
## iteration = 4000  theta = ( 0.1221661 0.5625419 1.467475 5.938889 )'
## iteration = 5000  theta = ( 0.1221661 0.5625419 1.467475 5.938889 )'
## iteration = 6000  theta = ( 0.1221661 0.5625419 1.467475 5.938889 )'
## iteration = 7000  theta = ( 0.1221661 0.5625419 1.467475 5.938889 )'
## iteration = 8000  theta = ( 0.1221661 0.5625419 1.467475 5.938889 )'
## iteration = 9000  theta = ( 0.1221661 0.5625419 1.467475 5.938889 )'
## iteration = 10000 theta = ( 0.1221661 0.5625419 1.467475 5.938889 )'

```

We see that the solution is 0.1221661,0.5625419,1.4674746,5.9388889.

Part (c)

Estimate the standard errors and pairwise correlations of your parameters, using any available method.

We have chosen to use the Bootstrap method.

```

# ns = (379,299,222,145,109,95,73,59,45,30,24,12,4,2,0,1,1)
# 379 responded 0 encounters
# 299 responded 1 encounters
# 222 responded 2 encounters
# ...
# 1 responded 16 encounters
#
# to resample, we resample from the data set that includes each
# persons response, as determined by ns.
data <- NULL
for (i in 1:length(ns))
{
  data <- append(data,rep((i-1),ns[i]))
}

make_into_counts <- function(data)
{
  ns <- NULL
  for (i in 0:16)
  {

```

```

    ni <- data[data == i]
    l <-length(ni)
    ns <- append(ns,l)
  }
  ns
}

m <- 10000 # bootstrap replicates
em_steps <- 1000
theta.bs <- em(theta,ns,em_steps,F)
thetas <- rbind(theta.bs)
for (i in 2:m)
{
  indices <- sample(N,N,replace=T)
  resampled <- make_into_counts(data[indices])
  theta.bs <- em(theta,resampled,em_steps,F)
  thetas <- rbind(thetas,theta.bs)
  if (i %% 100 == 0)
  {
    cat("iteration ", i, "\n")
    print(cov(thetas))
  }
}

## iteration 100
##           [,1]           [,2]           [,3]           [,4]
## [1,]  0.0005028137 -1.961295e-04  2.038906e-03  0.001983503
## [2,] -0.0001961295  4.630053e-04  3.703615e-05  0.001726600
## [3,]  0.0020389061  3.703615e-05  1.418904e-02  0.015610114
## [4,]  0.0019835030  1.726600e-03  1.561011e-02  0.043695794
## iteration 200
##           [,1]           [,2]           [,3]           [,4]
## [1,]  0.0004729425 -0.0001726064  0.0019356313  0.001803831
## [2,] -0.0001726064  0.0004203547  0.0001211203  0.001449850
## [3,]  0.0019356313  0.0001211203  0.0141049979  0.014342664
## [4,]  0.0018038311  0.0014498496  0.0143426640  0.040803483
## iteration 300
##           [,1]           [,2]           [,3]           [,4]
## [1,]  0.0004588332 -0.0001891526  0.0018081896  0.001630001
## [2,] -0.0001891526  0.0004216891  0.0000822698  0.001418845
## [3,]  0.0018081896  0.0000822698  0.0131616831  0.013889354
## [4,]  0.0016300006  0.0014188452  0.0138893543  0.040482575
## iteration 400
##           [,1]           [,2]           [,3]           [,4]
## [1,]  0.0004328737 -0.0001898423  0.001651652  0.001420829
## [2,] -0.0001898423  0.0004367875  0.000104603  0.001414274
## [3,]  0.0016516523  0.0001046030  0.012414708  0.012655730
## [4,]  0.0014208286  0.0014142744  0.012655730  0.037099430
## iteration 500
##           [,1]           [,2]           [,3]           [,4]
## [1,]  0.0004114875 -0.0001848842  0.0015432823  0.001287571
## [2,] -0.0001848842  0.0004533734  0.0001786003  0.001624405
## [3,]  0.0015432823  0.0001786003  0.0120886509  0.012939034
## [4,]  0.0012875713  0.0016244053  0.0129390341  0.038329060

```

```

## iteration 600
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004144797 -0.0001769493 0.001583010 0.001308955
## [2,] -0.0001769493 0.0004589919 0.000264984 0.001744639
## [3,] 0.0015830103 0.0002649840 0.012645907 0.013464524
## [4,] 0.0013089547 0.0017446395 0.013464524 0.038838679
## iteration 700
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004064432 -0.0001683966 0.0015700232 0.001329260
## [2,] -0.0001683966 0.0004571158 0.0002731392 0.001738693
## [3,] 0.0015700232 0.0002731392 0.0126133166 0.013300424
## [4,] 0.0013292599 0.0017386934 0.0133004243 0.038683222
## iteration 800
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004025111 -0.0001684245 0.0015761212 0.001330682
## [2,] -0.0001684245 0.0004710534 0.0002891239 0.001895352
## [3,] 0.0015761212 0.0002891239 0.0127025731 0.013852590
## [4,] 0.0013306822 0.0018953520 0.0138525901 0.040755349
## iteration 900
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004079241 -0.0001662936 0.0016151988 0.001495955
## [2,] -0.0001662936 0.0004771964 0.0003220499 0.001900824
## [3,] 0.0016151988 0.0003220499 0.0130940139 0.014881848
## [4,] 0.0014959549 0.0019008237 0.0148818484 0.042931403
## iteration 1000
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004264003 -0.0001677099 0.0017121911 0.001589926
## [2,] -0.0001677099 0.0004758569 0.0002804182 0.001856349
## [3,] 0.0017121911 0.0002804182 0.0133809315 0.015091281
## [4,] 0.0015899260 0.0018563492 0.0150912812 0.042551982
## iteration 1100
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004112396 -0.0001624904 0.001656796 0.001523592
## [2,] -0.0001624904 0.0004600818 0.000267179 0.001796693
## [3,] 0.0016567962 0.0002671790 0.013127004 0.014741970
## [4,] 0.0015235919 0.0017966932 0.014741970 0.042300016
## iteration 1200
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004065893 -0.0001625518 0.0016411870 0.001532955
## [2,] -0.0001625518 0.0004638276 0.0002825293 0.001824751
## [3,] 0.0016411870 0.0002825293 0.0131503899 0.014877687
## [4,] 0.0015329548 0.0018247515 0.0148776873 0.042412992
## iteration 1300
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004152746 -0.0001775105 0.0016486257 0.001515920
## [2,] -0.0001775105 0.0004775149 0.0002488204 0.001810461
## [3,] 0.0016486257 0.0002488204 0.0131860730 0.014832519
## [4,] 0.0015159198 0.0018104609 0.0148325191 0.042142682
## iteration 1400
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004116886 -0.0001759741 0.0016369124 0.001526861
## [2,] -0.0001759741 0.0004735076 0.0002407709 0.001784719
## [3,] 0.0016369124 0.0002407709 0.0130810918 0.014726705
## [4,] 0.0015268611 0.0017847190 0.0147267048 0.042066017

```

```

## iteration 1500
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004160885 -0.0001802945 0.0016476333 0.001529333
## [2,] -0.0001802945 0.0004755552 0.0002040224 0.001747531
## [3,] 0.0016476333 0.0002040224 0.0129128313 0.014373659
## [4,] 0.0015293335 0.0017475313 0.0143736589 0.041786667
## iteration 1600
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004162538 -0.0001811977 0.0016481748 0.001506189
## [2,] -0.0001811977 0.0004737466 0.0001938678 0.001775035
## [3,] 0.0016481748 0.0001938678 0.0128601728 0.014335166
## [4,] 0.0015061888 0.0017750350 0.0143351661 0.041905815
## iteration 1700
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004147575 -0.0001768437 0.001645373 0.001520817
## [2,] -0.0001768437 0.0004726278 0.000202936 0.001764351
## [3,] 0.0016453728 0.0002029360 0.012857612 0.014324334
## [4,] 0.0015208173 0.0017643511 0.014324334 0.041651832
## iteration 1800
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004139140 -0.0001734748 0.0016587757 0.001524464
## [2,] -0.0001734748 0.0004661266 0.0001985886 0.001741735
## [3,] 0.0016587757 0.0001985886 0.0129495567 0.014311501
## [4,] 0.0015244642 0.0017417348 0.0143115011 0.041580523
## iteration 1900
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004137828 -0.0001765014 0.0016627512 0.001497792
## [2,] -0.0001765014 0.0004690881 0.0001838348 0.001744307
## [3,] 0.0016627512 0.0001838348 0.0129328587 0.014164457
## [4,] 0.0014977923 0.0017443074 0.0141644566 0.041301861
## iteration 2000
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004145653 -0.0001771489 0.0016723886 0.001509032
## [2,] -0.0001771489 0.0004771632 0.0001939007 0.001780619
## [3,] 0.0016723886 0.0001939007 0.0130024155 0.014282044
## [4,] 0.0015090321 0.0017806188 0.0142820435 0.041611455
## iteration 2100
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004100603 -0.0001758606 0.0016441345 0.001484078
## [2,] -0.0001758606 0.0004742031 0.0001964646 0.001771077
## [3,] 0.0016441345 0.0001964646 0.0128438314 0.014100296
## [4,] 0.0014840782 0.0017710773 0.0141002963 0.041081664
## iteration 2200
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004094141 -0.0001782644 0.0016336989 0.001468098
## [2,] -0.0001782644 0.0004747682 0.0001919955 0.001773512
## [3,] 0.0016336989 0.0001919955 0.0127971628 0.014042097
## [4,] 0.0014680984 0.0017735116 0.0140420975 0.040915068
## iteration 2300
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004090371 -0.0001741259 0.001644988 0.001510316
## [2,] -0.0001741259 0.0004700629 0.000200275 0.001758669
## [3,] 0.0016449876 0.0002002750 0.012895977 0.014210853
## [4,] 0.0015103155 0.0017586691 0.014210853 0.041132620

```

```

## iteration 2400
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004092038 -0.0001754671 0.001634513 0.001506314
## [2,] -0.0001754671 0.0004711188 0.000190960 0.001736890
## [3,] 0.0016345127 0.0001909600 0.012784525 0.014045779
## [4,] 0.0015063139 0.0017368905 0.014045779 0.040693581
## iteration 2500
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004124026 -0.0001801872 0.0016443431 0.001510373
## [2,] -0.0001801872 0.0004775583 0.0001790191 0.001761709
## [3,] 0.0016443431 0.0001790191 0.0128236243 0.014111117
## [4,] 0.0015103731 0.0017617094 0.0141111166 0.041222659
## iteration 2600
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004147896 -0.0001835459 0.001641839 0.001500241
## [2,] -0.0001835459 0.0004805340 0.000174813 0.001753586
## [3,] 0.0016418394 0.0001748130 0.012801386 0.014103123
## [4,] 0.0015002414 0.0017535856 0.014103123 0.041189682
## iteration 2700
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004151804 -0.0001837420 0.0016472181 0.001496251
## [2,] -0.0001837420 0.0004770216 0.0001627393 0.001727997
## [3,] 0.0016472181 0.0001627393 0.0127993537 0.014006080
## [4,] 0.0014962511 0.0017279975 0.0140060795 0.040816164
## iteration 2800
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004132576 -0.0001838540 0.0016430303 0.001497506
## [2,] -0.0001838540 0.0004759374 0.0001538798 0.001728583
## [3,] 0.0016430303 0.0001538798 0.0127754597 0.013991486
## [4,] 0.0014975062 0.0017285825 0.0139914858 0.041020366
## iteration 2900
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004092355 -0.0001822372 0.001629384 0.001497349
## [2,] -0.0001822372 0.0004733033 0.000161304 0.001725782
## [3,] 0.0016293840 0.0001613040 0.012759191 0.014027567
## [4,] 0.0014973495 0.0017257823 0.014027567 0.040962228
## iteration 3000
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004085135 -0.0001821933 0.001628375 0.001501729
## [2,] -0.0001821933 0.0004757121 0.000165548 0.001729454
## [3,] 0.0016283752 0.0001655480 0.012799698 0.014045316
## [4,] 0.0015017289 0.0017294543 0.014045316 0.041067787
## iteration 3100
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004051823 -0.0001816412 0.0016080344 0.001480361
## [2,] -0.0001816412 0.0004772555 0.0001631984 0.001727813
## [3,] 0.0016080344 0.0001631984 0.0126610240 0.013886171
## [4,] 0.0014803613 0.0017278133 0.0138861710 0.040851042
## iteration 3200
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004087335 -0.0001828198 0.0016230440 0.001479894
## [2,] -0.0001828198 0.0004768770 0.0001575975 0.001714812
## [3,] 0.0016230440 0.0001575975 0.0127065216 0.013849319
## [4,] 0.0014798941 0.0017148121 0.0138493189 0.040650834

```

```

## iteration 3300
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004087427 -0.0001834090 0.0016230628 0.001474571
## [2,] -0.0001834090 0.0004779962 0.0001602443 0.001720960
## [3,] 0.0016230628 0.0001602443 0.0127122167 0.013877479
## [4,] 0.0014745710 0.0017209601 0.0138774789 0.040642047
## iteration 3400
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004075637 -0.0001823614 0.001623835 0.001486010
## [2,] -0.0001823614 0.0004751724 0.000162410 0.001692238
## [3,] 0.0016238350 0.0001624100 0.012730383 0.013916744
## [4,] 0.0014860097 0.0016922383 0.013916744 0.040498649
## iteration 3500
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004084971 -0.0001835128 0.001622077 0.001469057
## [2,] -0.0001835128 0.0004769958 0.000159973 0.001702859
## [3,] 0.0016220769 0.0001599730 0.012692176 0.013866264
## [4,] 0.0014690570 0.0017028590 0.013866264 0.040444667
## iteration 3600
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004082659 -0.0001842497 0.0016138087 0.001449978
## [2,] -0.0001842497 0.0004757468 0.0001605232 0.001710717
## [3,] 0.0016138087 0.0001605232 0.0126161803 0.013813744
## [4,] 0.0014499781 0.0017107171 0.0138137441 0.040434631
## iteration 3700
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004069987 -0.0001834077 0.0016120097 0.001436389
## [2,] -0.0001834077 0.0004753916 0.0001594311 0.001715579
## [3,] 0.0016120097 0.0001594311 0.0126127612 0.013755955
## [4,] 0.0014363889 0.0017155790 0.0137559552 0.040287122
## iteration 3800
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004052292 -0.0001850761 0.0015901821 0.001410502
## [2,] -0.0001850761 0.0004783086 0.0001661491 0.001722944
## [3,] 0.0015901821 0.0001661491 0.0125085573 0.013676635
## [4,] 0.0014105016 0.0017229440 0.0136766348 0.040233028
## iteration 3900
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004041565 -0.0001850811 0.0015867916 0.001406579
## [2,] -0.0001850811 0.0004783071 0.0001702557 0.001711088
## [3,] 0.0015867916 0.0001702557 0.0125044992 0.013660272
## [4,] 0.0014065785 0.0017110876 0.0136602722 0.040069546
## iteration 4000
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004055961 -0.0001861705 0.0015933096 0.001402686
## [2,] -0.0001861705 0.0004780842 0.0001606211 0.001697175
## [3,] 0.0015933096 0.0001606211 0.0125253180 0.013610010
## [4,] 0.0014026859 0.0016971751 0.0136100104 0.039830853
## iteration 4100
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004097737 -0.0001872491 0.0016178112 0.001431172
## [2,] -0.0001872491 0.0004768551 0.0001505465 0.001677706
## [3,] 0.0016178112 0.0001505465 0.0126664346 0.013726231
## [4,] 0.0014311717 0.0016777064 0.0137262311 0.039950837

```



```

## iteration 4200
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004076994 -0.0001858109 0.0016108760 0.001423726
## [2,] -0.0001858109 0.0004765038 0.0001545943 0.001683741
## [3,] 0.0016108760 0.0001545943 0.0126298969 0.013689341
## [4,] 0.0014237259 0.0016837408 0.0136893410 0.039991599
## iteration 4300
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004079266 -0.0001860367 0.0016107158 0.001415789
## [2,] -0.0001860367 0.0004751187 0.0001467721 0.001680319
## [3,] 0.0016107158 0.0001467721 0.0126060007 0.013627829
## [4,] 0.0014157892 0.0016803195 0.0136278289 0.039840804
## iteration 4400
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004055320 -0.0001844637 0.001600635 0.001406819
## [2,] -0.0001844637 0.0004734185 0.000151797 0.001673052
## [3,] 0.0016006354 0.0001517970 0.012544585 0.013536073
## [4,] 0.0014068186 0.0016730516 0.013536073 0.039553634
## iteration 4500
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004024451 -0.0001825676 0.0015891609 0.001400842
## [2,] -0.0001825676 0.0004719923 0.0001575463 0.001663446
## [3,] 0.0015891609 0.0001575463 0.0125203703 0.013511160
## [4,] 0.0014008418 0.0016634465 0.0135111601 0.039473829
## iteration 4600
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004006399 -0.0001827592 0.0015822006 0.001387907
## [2,] -0.0001827592 0.0004714602 0.0001539083 0.001658098
## [3,] 0.0015822006 0.0001539083 0.0124881800 0.013430109
## [4,] 0.0013879068 0.0016580984 0.0134301086 0.039356703
## iteration 4700
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004019046 -0.0001826581 0.0015887047 0.001395022
## [2,] -0.0001826581 0.0004710381 0.0001518507 0.001664616
## [3,] 0.0015887047 0.0001518507 0.0125045559 0.013477765
## [4,] 0.0013950217 0.0016646164 0.0134777646 0.039503214
## iteration 4800
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004038823 -0.0001845470 0.0015928723 0.001399236
## [2,] -0.0001845470 0.0004734607 0.0001529025 0.001662396
## [3,] 0.0015928723 0.0001529025 0.0125413206 0.013522956
## [4,] 0.0013992361 0.0016623956 0.0135229558 0.039450327
## iteration 4900
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004012205 -0.0001833171 0.0015830454 0.001388511
## [2,] -0.0001833171 0.0004732196 0.0001572175 0.001661530
## [3,] 0.0015830454 0.0001572175 0.0125019373 0.013461672
## [4,] 0.0013885114 0.0016615298 0.0134616724 0.039241116
## iteration 5000
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004016326 -0.0001831367 0.0015841049 0.001385851
## [2,] -0.0001831367 0.0004728952 0.0001589137 0.001665706
## [3,] 0.0015841049 0.0001589137 0.0124892790 0.013444865
## [4,] 0.0013858506 0.0016657057 0.0134448651 0.039293620

```

```

## iteration 5100
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004015419 -0.0001840544  0.0015817741  0.001378996
## [2,] -0.0001840544  0.0004737531  0.0001587592  0.001673454
## [3,]  0.0015817741  0.0001587592  0.0124908998  0.013452481
## [4,]  0.0013789963  0.0016734543  0.0134524813  0.039363174
## iteration 5200
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004038412 -0.0001841862  0.0015913023  0.001385730
## [2,] -0.0001841862  0.0004731221  0.0001584955  0.001666397
## [3,]  0.0015913023  0.0001584955  0.0125134432  0.013467872
## [4,]  0.0013857296  0.0016663970  0.0134678724  0.039338578
## iteration 5300
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004040938 -0.0001861660  0.0015856913  0.001371768
## [2,] -0.0001861660  0.0004739526  0.0001541198  0.001664283
## [3,]  0.0015856913  0.0001541198  0.0124848638  0.013420429
## [4,]  0.0013717682  0.0016642833  0.0134204287  0.039200196
## iteration 5400
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004024244 -0.0001854621  0.0015777338  0.001362838
## [2,] -0.0001854621  0.0004756834  0.0001553037  0.001672207
## [3,]  0.0015777338  0.0001553037  0.0124271830  0.013371255
## [4,]  0.0013628378  0.0016722071  0.0133712555  0.039166990
## iteration 5500
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004009885 -0.0001851274  0.0015699803  0.001358070
## [2,] -0.0001851274  0.0004778723  0.0001633452  0.001693008
## [3,]  0.0015699803  0.0001633452  0.0124026021  0.013401944
## [4,]  0.0013580700  0.0016930078  0.0134019444  0.039305997
## iteration 5600
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004001053 -0.0001848129  0.0015684084  0.001354829
## [2,] -0.0001848129  0.0004770884  0.0001609486  0.001688578
## [3,]  0.0015684084  0.0001609486  0.0124019402  0.013395083
## [4,]  0.0013548287  0.0016885778  0.0133950826  0.039260302
## iteration 5700
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004017671 -0.0001859207  0.0015756177  0.001361676
## [2,] -0.0001859207  0.0004765438  0.0001514264  0.001677572
## [3,]  0.0015756177  0.0001514264  0.0124124193  0.013397192
## [4,]  0.0013616765  0.0016775715  0.0133971922  0.039291816
## iteration 5800
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004005046 -0.0001858364  0.0015731130  0.001364359
## [2,] -0.0001858364  0.0004761375  0.0001483154  0.001664534
## [3,]  0.0015731130  0.0001483154  0.0124247473  0.013426787
## [4,]  0.0013643590  0.0016645337  0.0134267871  0.039315978
## iteration 5900
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004013137 -0.0001867581  0.0015766540  0.001371971
## [2,] -0.0001867581  0.0004767379  0.0001484983  0.001660266
## [3,]  0.0015766540  0.0001484983  0.0124733576  0.013503808
## [4,]  0.0013719712  0.0016602656  0.0135038083  0.039424246

```

```

## iteration 6000
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004014365 -0.0001874036  0.0015784918  0.001376795
## [2,] -0.0001874036  0.0004773699  0.0001441186  0.001649991
## [3,]  0.0015784918  0.0001441186  0.0124742258  0.013501774
## [4,]  0.0013767945  0.0016499907  0.0135017740  0.039365021
## iteration 6100
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004020162 -0.0001880096  0.001583586  0.001372243
## [2,] -0.0001880096  0.0004789531  0.000143932  0.001658114
## [3,]  0.0015835864  0.0001439320  0.012511751  0.013488050
## [4,]  0.0013722428  0.0016581144  0.013488050  0.039336863
## iteration 6200
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004018746 -0.0001879583  0.0015845242  0.001367621
## [2,] -0.0001879583  0.0004806377  0.0001504711  0.001666224
## [3,]  0.0015845242  0.0001504711  0.0125371267  0.013524114
## [4,]  0.0013676208  0.0016662244  0.0135241141  0.039406388
## iteration 6300
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004023624 -0.0001876180  0.0015847096  0.001370915
## [2,] -0.0001876180  0.0004796021  0.0001485523  0.001659760
## [3,]  0.0015847096  0.0001485523  0.0124984226  0.013499905
## [4,]  0.0013709152  0.0016597599  0.0134999054  0.039424508
## iteration 6400
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004024437 -0.0001877503  0.0015857778  0.001377654
## [2,] -0.0001877503  0.0004790853  0.0001441742  0.001649743
## [3,]  0.0015857778  0.0001441742  0.0125009327  0.013510056
## [4,]  0.0013776544  0.0016497432  0.0135100558  0.039460129
## iteration 6500
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004027450 -0.0001874346  0.0015886791  0.001385181
## [2,] -0.0001874346  0.0004774763  0.0001412945  0.001634831
## [3,]  0.0015886791  0.0001412945  0.0125053977  0.013514986
## [4,]  0.0013851809  0.0016348312  0.0135149865  0.039435550
## iteration 6600
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004015398 -0.0001871316  0.0015828350  0.001380264
## [2,] -0.0001871316  0.0004772634  0.0001437703  0.001636516
## [3,]  0.0015828350  0.0001437703  0.0124842141  0.013508696
## [4,]  0.0013802642  0.0016365164  0.0135086959  0.039402410
## iteration 6700
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004034479 -0.0001881143  0.0015905586  0.001393490
## [2,] -0.0001881143  0.0004776668  0.0001388635  0.001632291
## [3,]  0.0015905586  0.0001388635  0.0125249545  0.013592576
## [4,]  0.0013934901  0.0016322908  0.0135925758  0.039520840
## iteration 6800
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004023847 -0.0001872736  0.0015847070  0.001389808
## [2,] -0.0001872736  0.0004771731  0.0001392261  0.001626599
## [3,]  0.0015847070  0.0001392261  0.0124670392  0.013527465
## [4,]  0.0013898084  0.0016265994  0.0135274650  0.039385182

```

```

## iteration 6900
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004021607 -0.0001870602  0.0015843893  0.001389272
## [2,] -0.0001870602  0.0004770252  0.0001408231  0.001630719
## [3,]  0.0015843893  0.0001408231  0.0124683946  0.013527624
## [4,]  0.0013892721  0.0016307190  0.0135276241  0.039410044
## iteration 7000
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004029344 -0.0001872561  0.0015895252  0.001397475
## [2,] -0.0001872561  0.0004758039  0.0001354844  0.001618447
## [3,]  0.0015895252  0.0001354844  0.0124829029  0.013540795
## [4,]  0.0013974753  0.0016184466  0.0135407951  0.039375557
## iteration 7100
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004026963 -0.0001869715  0.0015884793  0.001403528
## [2,] -0.0001869715  0.0004765544  0.0001446278  0.001628991
## [3,]  0.0015884793  0.0001446278  0.0124929345  0.013617781
## [4,]  0.0014035283  0.0016289911  0.0136177815  0.039464041
## iteration 7200
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004035843 -0.0001873946  0.0015938318  0.001411687
## [2,] -0.0001873946  0.0004764959  0.0001379966  0.001623168
## [3,]  0.0015938318  0.0001379966  0.0125083730  0.013642576
## [4,]  0.0014116874  0.0016231683  0.0136425762  0.039524791
## iteration 7300
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004043350 -0.0001877552  0.0015974552  0.001415163
## [2,] -0.0001877552  0.0004749961  0.0001328922  0.001613713
## [3,]  0.0015974552  0.0001328922  0.0125252118  0.013643061
## [4,]  0.0014151634  0.0016137134  0.0136430607  0.039509153
## iteration 7400
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004032570 -0.0001868522  0.0015952332  0.001416507
## [2,] -0.0001868522  0.0004741537  0.0001350029  0.001613628
## [3,]  0.0015952332  0.0001350029  0.0125219919  0.013653422
## [4,]  0.0014165069  0.0016136276  0.0136534222  0.039556849
## iteration 7500
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004030561 -0.0001856412  0.001595910  0.001418078
## [2,] -0.0001856412  0.0004732598  0.000139549  0.001615703
## [3,]  0.0015959098  0.0001395490  0.012521325  0.013649411
## [4,]  0.0014180782  0.0016157028  0.013649411  0.039556193
## iteration 7600
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004041606 -0.0001863164  0.0015979701  0.001422733
## [2,] -0.0001863164  0.0004733558  0.0001379977  0.001605416
## [3,]  0.0015979701  0.0001379977  0.0125193898  0.013636098
## [4,]  0.0014227326  0.0016054164  0.0136360977  0.039457181
## iteration 7700
##          [,1]          [,2]          [,3]          [,4]
## [1,]  0.0004043309 -0.0001853273  0.0015998365  0.001420228
## [2,] -0.0001853273  0.0004723620  0.0001415232  0.001604015
## [3,]  0.0015998365  0.0001415232  0.0125292055  0.013627229
## [4,]  0.0014202277  0.0016040152  0.0136272290  0.039486347

```

```

## iteration 7800
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004048504 -0.0001847713 0.0016052438 0.001430729
## [2,] -0.0001847713 0.0004717630 0.0001402141 0.001604219
## [3,] 0.0016052438 0.0001402141 0.0125675728 0.013678280
## [4,] 0.0014307294 0.0016042192 0.0136782800 0.039536603
## iteration 7900
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004044445 -0.0001848540 0.0016042077 0.001429972
## [2,] -0.0001848540 0.0004733111 0.0001395748 0.001610083
## [3,] 0.0016042077 0.0001395748 0.0125628020 0.013690279
## [4,] 0.0014299717 0.0016100826 0.0136902788 0.039591338
## iteration 8000
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004043154 -0.0001845321 0.0016039552 0.001430818
## [2,] -0.0001845321 0.0004729283 0.0001419234 0.001612323
## [3,] 0.0016039552 0.0001419234 0.0125677305 0.013714288
## [4,] 0.0014308185 0.0016123226 0.0137142882 0.039587137
## iteration 8100
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004040699 -0.0001842702 0.0016040908 0.001429402
## [2,] -0.0001842702 0.0004720087 0.0001426526 0.001606520
## [3,] 0.0016040908 0.0001426526 0.0125765480 0.013712083
## [4,] 0.0014294022 0.0016065200 0.0137120829 0.039559249
## iteration 8200
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004043834 -0.0001853009 0.0016024607 0.001429709
## [2,] -0.0001853009 0.0004721168 0.0001376129 0.001597725
## [3,] 0.0016024607 0.0001376129 0.0125441800 0.013696631
## [4,] 0.0014297086 0.0015977253 0.0136966308 0.039513020
## iteration 8300
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004057798 -0.0001861451 0.0016067057 0.001437197
## [2,] -0.0001861451 0.0004725805 0.0001334232 0.001594781
## [3,] 0.0016067057 0.0001334232 0.0125565585 0.013713902
## [4,] 0.0014371972 0.0015947810 0.0137139022 0.039507454
## iteration 8400
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004060145 -0.0001858334 0.0016084032 0.001441968
## [2,] -0.0001858334 0.0004729303 0.0001375857 0.001596390
## [3,] 0.0016084032 0.0001375857 0.0125662155 0.013740960
## [4,] 0.0014419681 0.0015963899 0.0137409597 0.039581185
## iteration 8500
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004057525 -0.0001859895 0.0016105815 0.001437960
## [2,] -0.0001859895 0.0004726667 0.0001352539 0.001597193
## [3,] 0.0016105815 0.0001352539 0.0125916360 0.013742167
## [4,] 0.0014379602 0.0015971933 0.0137421673 0.039537256
## iteration 8600
##      [,1]      [,2]      [,3]      [,4]
## [1,] 0.0004060114 -0.0001863734 0.0016119841 0.001435234
## [2,] -0.0001863734 0.0004729652 0.0001340095 0.001597387
## [3,] 0.0016119841 0.0001340095 0.0126117774 0.013746036
## [4,] 0.0014352342 0.0015973872 0.0137460361 0.039499836

```

```

## iteration 8700
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004062383 -0.0001860263 0.0016147117 0.001440800
## [2,] -0.0001860263 0.0004736472 0.0001377228 0.001605681
## [3,] 0.0016147117 0.0001377228 0.0126359306 0.013786082
## [4,] 0.0014407996 0.0016056811 0.0137860821 0.039634493
## iteration 8800
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004063572 -0.0001865120 0.0016148427 0.001434323
## [2,] -0.0001865120 0.0004740701 0.0001351495 0.001599798
## [3,] 0.0016148427 0.0001351495 0.0126228914 0.013726480
## [4,] 0.0014343229 0.0015997982 0.0137264803 0.039550769
## iteration 8900
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004069336 -0.0001862376 0.0016174601 0.001443223
## [2,] -0.0001862376 0.0004738720 0.0001373237 0.001605670
## [3,] 0.0016174601 0.0001373237 0.0126423051 0.013784024
## [4,] 0.0014432228 0.0016056697 0.0137840238 0.039703025
## iteration 9000
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004074698 -0.0001859218 0.0016224399 0.001450143
## [2,] -0.0001859218 0.0004738131 0.0001372384 0.001603230
## [3,] 0.0016224399 0.0001372384 0.0126614188 0.013790333
## [4,] 0.0014501425 0.0016032300 0.0137903333 0.039681546
## iteration 9100
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004078906 -0.0001866510 0.0016224993 0.001444933
## [2,] -0.0001866510 0.0004741105 0.0001344245 0.001597423
## [3,] 0.0016224993 0.0001344245 0.0126452109 0.013738939
## [4,] 0.0014449330 0.0015974227 0.0137389387 0.039559868
## iteration 9200
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004092736 -0.0001869957 0.0016292050 0.001448660
## [2,] -0.0001869957 0.0004734179 0.0001306551 0.001590535
## [3,] 0.0016292050 0.0001306551 0.0126676840 0.013742651
## [4,] 0.0014486603 0.0015905353 0.0137426511 0.039533660
## iteration 9300
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004087596 -0.0001872841 0.0016259126 0.001443004
## [2,] -0.0001872841 0.0004747982 0.0001321641 0.001599979
## [3,] 0.0016259126 0.0001321641 0.0126520639 0.013725366
## [4,] 0.0014430042 0.0015999795 0.0137253661 0.039543783
## iteration 9400
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004085071 -0.0001877796 0.0016235818 0.001438365
## [2,] -0.0001877796 0.0004761875 0.0001341946 0.001612360
## [3,] 0.0016235818 0.0001341946 0.0126403633 0.013719831
## [4,] 0.0014383654 0.0016123605 0.0137198308 0.039596062
## iteration 9500
##          [,1]          [,2]          [,3]          [,4]
## [1,] 0.0004082129 -0.0001869858 0.0016234140 0.001444267
## [2,] -0.0001869858 0.0004755978 0.0001359561 0.001611437
## [3,] 0.0016234140 0.0001359561 0.0126379757 0.013742128
## [4,] 0.0014442675 0.0016114369 0.0137421284 0.039592424

```

```
## iteration 9600
##           [,1]           [,2]           [,3]           [,4]
## [1,]  0.0004091573 -0.0001869738  0.0016271057  0.001452470
## [2,] -0.0001869738  0.0004763240  0.0001370514  0.001618108
## [3,]  0.0016271057  0.0001370514  0.0126528092  0.013791380
## [4,]  0.0014524705  0.0016181082  0.0137913795  0.039696093
## iteration 9700
##           [,1]           [,2]           [,3]           [,4]
## [1,]  0.0004086478 -0.0001860897  0.0016250056  0.001453139
## [2,] -0.0001860897  0.0004757763  0.0001419182  0.001619964
## [3,]  0.0016250056  0.0001419182  0.0126489356  0.013791685
## [4,]  0.0014531394  0.0016199636  0.0137916852  0.039677192
## iteration 9800
##           [,1]           [,2]           [,3]           [,4]
## [1,]  0.0004083653 -0.0001859143  0.0016227417  0.001453750
## [2,] -0.0001859143  0.0004758347  0.0001436156  0.001619215
## [3,]  0.0016227417  0.0001436156  0.0126366806  0.013785264
## [4,]  0.0014537495  0.0016192148  0.0137852644  0.039679339
## iteration 9900
##           [,1]           [,2]           [,3]           [,4]
## [1,]  0.0004081208 -0.0001851963  0.0016244520  0.001453281
## [2,] -0.0001851963  0.0004758142  0.0001454919  0.001619941
## [3,]  0.0016244520  0.0001454919  0.0126487037  0.013774860
## [4,]  0.0014532806  0.0016199410  0.0137748602  0.039658983
## iteration 10000
##           [,1]           [,2]           [,3]           [,4]
## [1,]  0.0004076668 -0.0001852179  0.0016217490  0.001450841
## [2,] -0.0001852179  0.0004760241  0.0001453765  0.001622406
## [3,]  0.0016217490  0.0001453765  0.0126274484  0.013746974
## [4,]  0.0014508411  0.0016224055  0.0137469738  0.039704028

cov.bs <- cov(thetas)
cor.bs <- cor(thetas)
```

The Bootstrap estimator of the covariance matrix is given by

```
##           [,1]           [,2]           [,3]           [,4]
## [1,]  0.0004076668 -0.0001852179  0.0016217490  0.001450841
## [2,] -0.0001852179  0.0004760241  0.0001453765  0.001622406
## [3,]  0.0016217490  0.0001453765  0.0126274484  0.013746974
## [4,]  0.0014508411  0.0016224055  0.0137469738  0.039704028
```

and the correlation matrix is given by

```
##           [,1]           [,2]           [,3]           [,4]
## [1,]  1.00000000 -0.42045155  0.71478130  0.3606201
## [2,] -0.4204516  1.00000000  0.05929554  0.3731880
## [3,]  0.7147813  0.05929554  1.00000000  0.6139486
## [4,]  0.3606201  0.37318801  0.61394861  1.0000000
```

Let's try using the Hessian of the observed information matrix.

```
library(numDeriv)
loglike <- function(theta)
{
  s <- 0
  for (x in data)
```

```

{
  s <- s + log(theta[1]*as.numeric(x==0) +
               theta[2]*dpois(x,theta[3]) +
               (1-theta[1]-theta[2])*dpois(x,theta[4]))
}
s
}
mle <- c(0.1221661,0.5625419,1.4674746,5.9388889)
solve(-hessian(loglike,mle))

```

```

##           [,1]           [,2]           [,3]           [,4]
## [1,]  0.0003799048 -1.909698e-04  1.438556e-03  0.001184057
## [2,] -0.0001909698  4.657702e-04  7.132638e-05  0.001417409
## [3,]  0.0014385560  7.132638e-05  1.111722e-02  0.011376017
## [4,]  0.0011840568  1.417409e-03  1.137602e-02  0.034664940

```